

Assignment

Title: Bank Customer Analytics using Feature Engineering

Introduction

Banks generate large volumes of customer data during loan applications, including information such as income, age, credit score, employment status, loan amount, existing loans, and repayment history. Analyzing this data effectively helps financial institutions identify customers who are more likely to repay their loans and those who may default. Early prediction of loan default enables banks to reduce financial risk, improve decision-making, and provide better credit management.

In machine learning, feature engineering and dimensionality reduction play a significant role in improving predictive performance. Feature engineering creates new informative features from existing data, while feature selection techniques identify the most relevant variables by eliminating unnecessary or redundant information. Dimensionality reduction methods such as Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA) further simplify the dataset while preserving important information, resulting in faster model training and improved efficiency.

This assignment focuses on developing an optimized dataset for bank customer analytics by applying feature engineering, feature selection methods (Filter, Wrapper, and Embedded), and dimensionality reduction techniques (PCA and LDA). A Logistic Regression classifier is then trained on the transformed data to predict loan default status. The performance of different approaches is compared using evaluation metrics such as Accuracy, Precision, Recall, and F1-Score to determine the most effective technique for loan default prediction.

Problem Statement

A commercial bank wants to reduce loan defaults by identifying the most significant customer features that predict repayment risk. This assignment develops an optimized, low-dimensional dataset using feature engineering, feature selection and dimensionality reduction, and applies it to a classification model that predicts whether a customer will default on a loan.

Objectives

- Create meaningful engineered features from raw banking data.
- Apply and compare Filter, Wrapper and Embedded feature selection methods.
- Reduce dimensionality using PCA and LDA and compare their effect on model performance.

Technology Stack

Component	Details
Dataset Source	bank_customer_data.csv (provided customer records)
Programming Language	Python 3
Development Environment	Jupyter / Python script (executed via command line)
ML Libraries	scikit-learn (feature selection, PCA, LDA, Logistic Regression, Random Forest, GridSearchCV)
Data Handling	pandas, numpy
Visualization Tools	matplotlib, seaborn
Deployment Tools	Not applicable — prediction demonstrated via script (Section 9)

Dataset Description

Attribute	Value
Dataset Source	bank_customer_data.csv
Number of Records (raw)	20,025
Number of Records (after cleaning)	19,900
Number of Attributes (raw)	8 (7 features + 1 target)
Target Variable	Default_Status (0 = No Default, 1 = Default)
Class Balance	No Default: 15,434 Default: 4,591 (~23% default rate — imbalanced)
Data Type	Mixed — numeric (Income, Age, Credit_Score, Loan_Amount, Existing_Loans) and categorical (Employment_Status, Repayment_History)

Feature Description:

- Income — annual income of the customer (numeric)
- Age — customer age in years (numeric)
- Credit_Score — credit bureau score (numeric)

- Loan_Amount — amount of loan requested (numeric)
- Employment_Status — Employed / Self-Employed / Retired / Unemployed (categorical)
- Existing_Loans — count of loans already held (numeric)
- Repayment_History — Good / Average / Poor (categorical)
- Default_Status — target label: whether the customer defaulted

Data Preprocessing

Step	Method Applied
Missing Values	Income and Credit_Score imputed with column median; Employment_Status imputed with mode
Data Cleaning	Duplicate rows dropped; records with non-positive Loan_Amount removed
Outlier Detection	IQR method applied to Income and Loan_Amount; values clipped to $[Q1-1.5*IQR, Q3+1.5*IQR]$
Data Transformation	Log transform ($\log_{10}$) applied to Income to reduce right-skew
Encoding	One-hot encoding (drop-first) applied to Employment_Status and Repayment_History
Feature Scaling	StandardScaler applied before PCA, LDA and RFE
Train-Test Split	80/20 stratified split on Default_Status, random_state=42

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
df = pd.read_csv("bank_customer_data.csv")
df["Income"] = df["Income"].fillna(df["Income"].median())
df["Credit_Score"] = df["Credit_Score"].fillna(df["Credit_Score"].median())
df["Employment_Status"] = df["Employment_Status"].fillna(
    df["Employment_Status"].mode()[0]
)
df = df.drop_duplicates()
df = df[df["Loan_Amount"] > 0]
df["Debt_to_Income"] = df["Loan_Amount"] / (df["Income"] + 1)
df["Loan_to_CreditScore"] = df["Loan_Amount"] / (df["Credit_Score"] + 1)
df["Loan_Burden"] = df["Existing_Loans"] * df["Loan_Amount"] / 1000
df["Income"] = np.log1p(df["Income"])
df = pd.get_dummies(
    df,
    columns=["Employment_Status", "Repayment_History"],
    drop_first=True
)
X = df.drop("Default_Status", axis=1)
y = df["Default_Status"]
selector = SelectKBest(f_classif, k=min(8, X.shape[1]))
selector.fit(X, y)
filter_features = X.columns[selector.get_support()]
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)
importance = pd.Series(
    rf.feature_importances_,
    index=X.columns
).sort_values(ascending=False)
embedded_features = importance.head(8).index
plt.figure(figsize=(8, 5))
importance.head(10).sort_values().plot(kind="barh")
plt.title("Random Forest Feature Importance")
plt.tight_layout()
plt.savefig("feature_importance.png")
```

```
plt.show()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=0.95)
X_pca = pca.fit_transform(X_scaled)
plt.figure(figsize=(7, 5))
plt.plot(
    np.cumsum(PCA().fit(X_scaled).explained_variance_ratio_),
    marker="o"
)
plt.axhline(0.95, linestyle="--")
plt.title("PCA Explained Variance")
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Variance")
plt.tight_layout()
plt.savefig("pca_variance.png")
plt.show()
lda = LinearDiscriminantAnalysis(n_components=1)
X_lda = lda.fit_transform(X_scaled, y)
plt.figure(figsize=(7, 5))
sns.histplot(x=X_lda.ravel(), hue=y, bins=30, kde=True)
plt.title("LDA Class Separation")
plt.xlabel("LDA Component")
plt.tight_layout()
plt.savefig("lda_distribution.png")
plt.show()
def evaluate(X_data):
    X_train, X_test, y_train, y_test = train_test_split(
        X_data, y, test_size=0.2, stratify=y, random_state=42
    )
    model = LogisticRegression(max_iter=2000)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    return [
        accuracy_score(y_test, pred),
```

```
precision_score(y_test, pred, zero_division=0),
recall_score(y_test, pred, zero_division=0),
f1_score(y_test, pred, zero_division=0)
]
results = pd.DataFrame(
    [
        evaluate(X_scaled),
        evaluate(X_pca),
        evaluate(X_lda)
    ],
    index=["All Features", "PCA", "LDA"],
    columns=["Accuracy", "Precision", "Recall", "F1"]
)
print(results.round(3))
results.plot(kind="bar", figsize=(8, 5))
plt.title("All Features vs PCA vs LDA")
plt.ylabel("Score")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig("model_comparison.png")
plt.show()

# Final LDA + Logistic Regression model
X_train, X_test, y_train, y_test = train_test_split(
    X_lda, y, test_size=0.2, stratify=y, random_state=42
)
grid = GridSearchCV(
    LogisticRegression(max_iter=2000),
    {"C": [0.01, 0.1, 1, 10]},
    cv=5
)
grid.fit(X_train, y_train)
prediction = grid.predict(X_test)
print("\nBest C:", grid.best_params_["C"])
print("Accuracy:", accuracy_score(y_test, prediction))
```

```
print("Precision:", precision_score(y_test, prediction, zero_division=0))
print("Recall:", recall_score(y_test, prediction, zero_division=0))
print("F1 Score:", f1_score(y_test, prediction, zero_division=0))
cm = confusion_matrix(y_test, prediction)
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Default", "Default"],
    yticklabels=["No Default", "Default"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.savefig("confusion_matrix.png")
plt.show()
new_customer = pd.DataFrame([
    {
        col: 0 for col in X.columns
    }
])
new_customer["Income"] = np.log1p(45000)
new_customer["Age"] = 34
new_customer["Credit_Score"] = 610
new_customer["Loan_Amount"] = 250000
new_customer["Existing_Loans"] = 2
new_customer["Debt_to_Income"] = 250000 / (np.log1p(45000) + 1)
new_customer["Loan_to_CreditScore"] = 250000 / 611
new_customer["Loan_Burden"] = 500
new_lda = lda.transform(scaler.transform(new_customer))
result = grid.predict(new_lda)
print(
    "\nNew Customer:",
    "Default" if result[0] == 1 else "No Default"
)
```

Model Development

Derived Features

- $\text{Debt_to_Income} = \text{Loan_Amount} / (\text{Income} + 1)$
- $\text{Loan_to_CreditScore} = \text{Loan_Amount} / (\text{Credit_Score} + 1)$
- $\text{Loan_Burden} = \text{Existing_Loans} * \text{Loan_Amount} / 1000$

After encoding, the engineered feature set contains 13 predictors.

Feature Selection — Filter Method (ANOVA F-test)

SelectKBest with the ANOVA F-statistic was used to rank features by their individual relationship with the target. Top 8 selected: Credit_Score, Loan_Amount, Debt_to_Income, Loan_to_CreditScore, Loan_Burden, Employment_Status_Unemployed, Repayment_History_Good, Repayment_History_Poor.

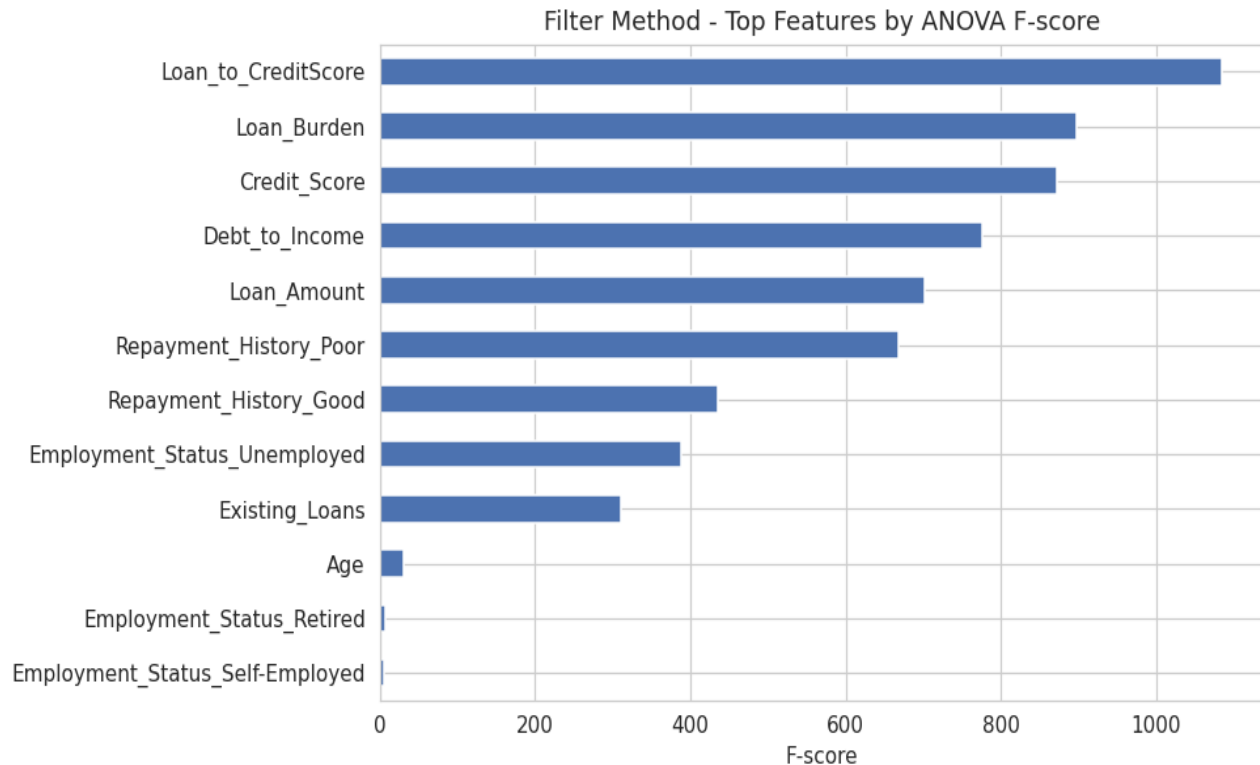


Figure 1: Filter Method — Top features ranked by ANOVA F-score

Feature Selection — Wrapper Method (RFE)

Recursive Feature Elimination with Logistic Regression selected: Income, Credit_Score, Loan_Amount, Existing_Loans, Debt_to_Income, Employment_Status_Unemployed, Repayment_History_Good, Repayment_History_Poor.

Feature Selection — Embedded Method (Random Forest Importance)

A Random Forest classifier (200 trees) ranked features by Gini importance. Top 8: Income, Credit_Score, Loan_to_CreditScore, Debt_to_Income, Loan_Burden, Loan_Amount, Age, Existing_Loans.

Interpretation of Feature Selection Results

The results obtained from the Wrapper and Embedded feature selection methods indicate that Income, Credit_Score, Loan_Amount, and Debt_to_Income are among the most influential variables for predicting loan default. The Wrapper method selected features that maximize the performance of the Logistic Regression model, while the Embedded method ranked features based on their importance during Random Forest model training. The presence of several common features across both methods demonstrates their strong predictive capability and confirms their importance in building an accurate loan default prediction model.

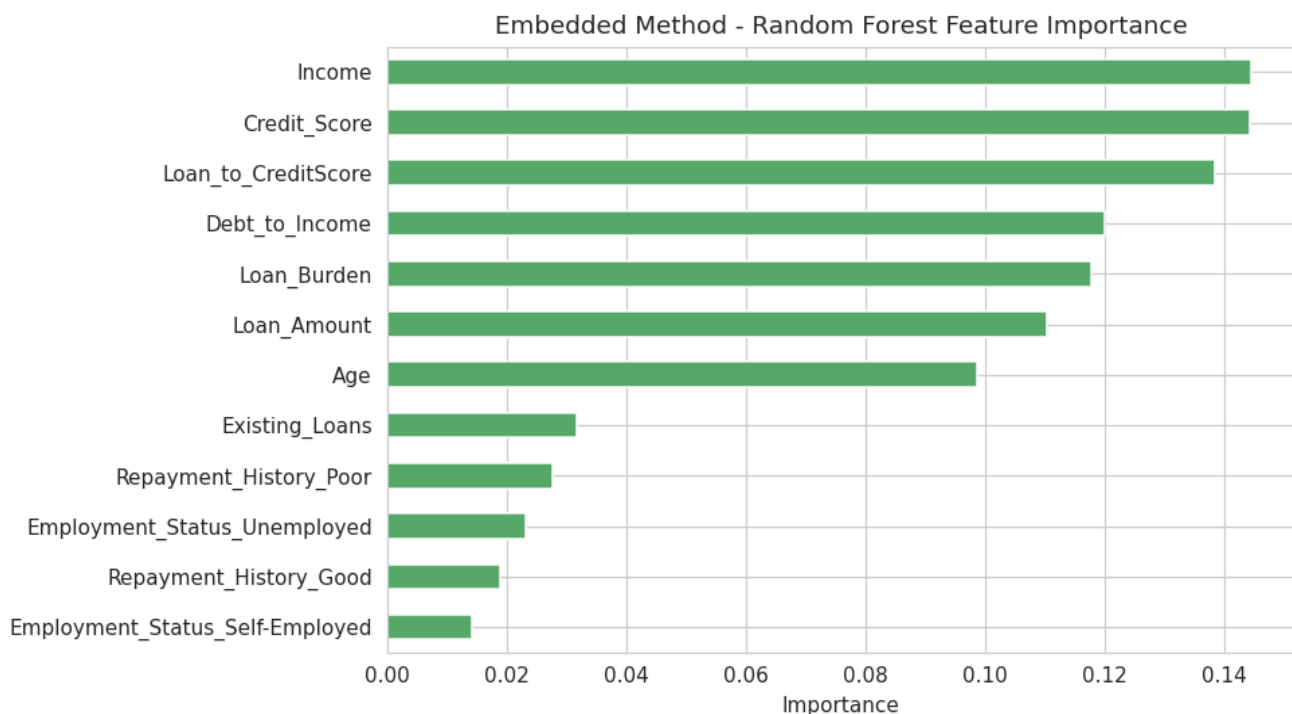


Figure 2: Embedded Method — Random Forest feature importance

Comparison of Feature Selection Methods

Credit_Score, Debt_to_Income and Loan_Amount were selected by all three methods, indicating they are the most robust predictors of default.

Feature Selection Method Comparison (1 = Selected)			
	Filter	Wrapper	Embedded
Age	0	0	1
Credit_Score	1	1	1
Debt_to_Income	1	1	1
Employment_Status_Unemployed	1	1	0
Existing_Loans	0	1	1
Income	0	1	1
Loan_Amount	1	1	1
Loan_Burden	1	0	1
Loan_to_CreditScore	1	0	1
Repayment_History_Good	1	1	0
Repayment_History_Poor	1	1	0

Figure 3: Feature selection overlap across Filter, Wrapper and Embedded methods

Principal Component Analysis (PCA)

PCA was applied to the standardized feature set. 9 of the 13 original components are required to retain 95% of the variance, showing that the engineered feature set carries limited redundancy.

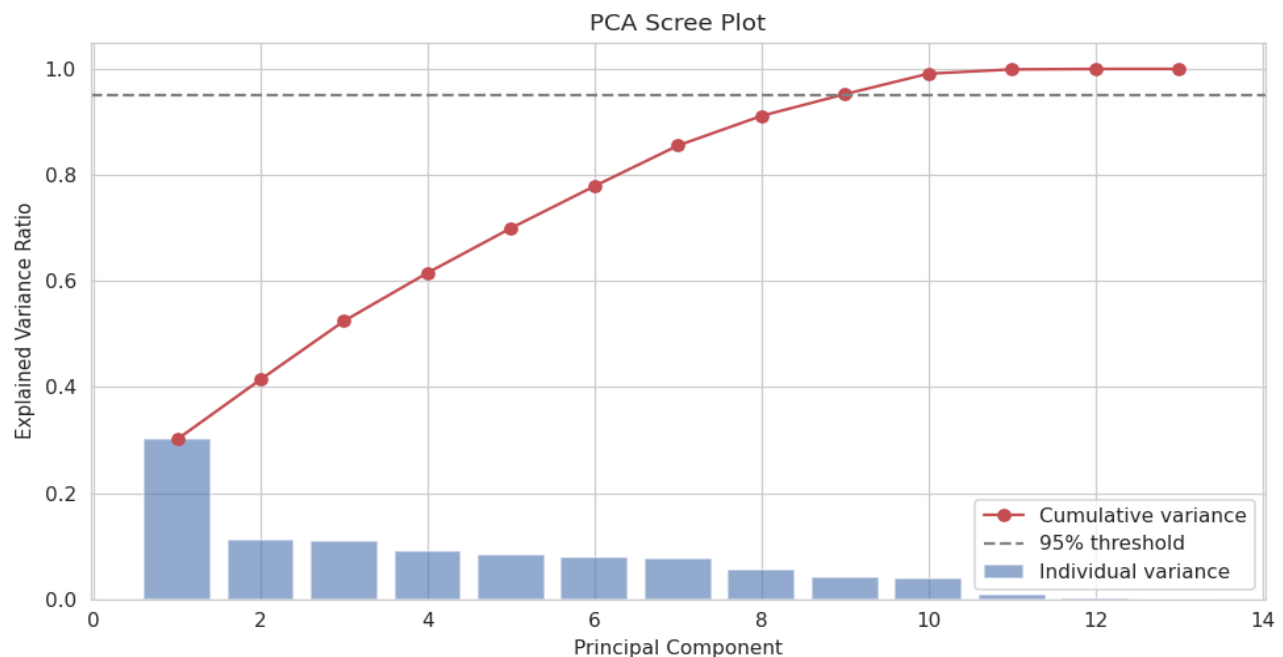


Figure 4: PCA scree plot — individual and cumulative explained variance

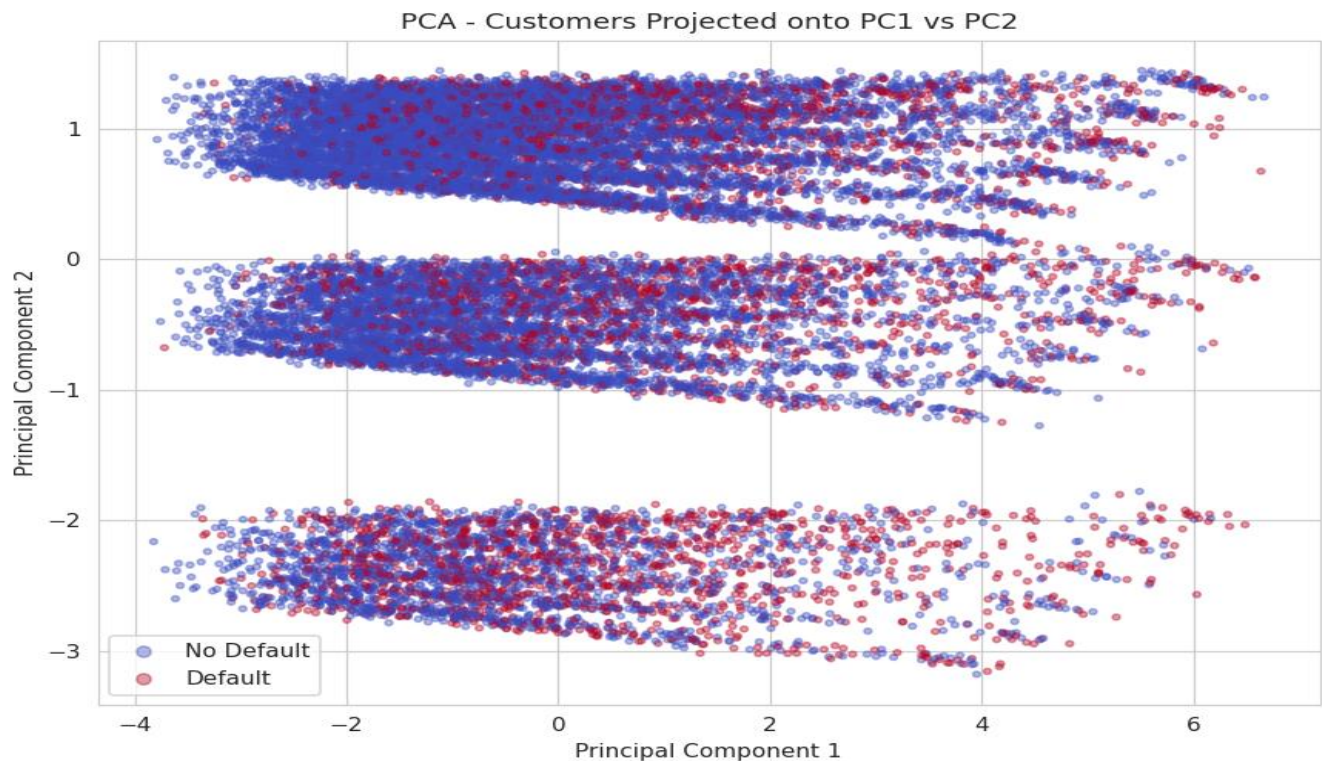


Figure 5: Customers projected onto the first two principal components

Linear Discriminant Analysis (LDA)

Since this is a binary classification problem, LDA reduces the feature space to a single discriminant component (max components = classes - 1).

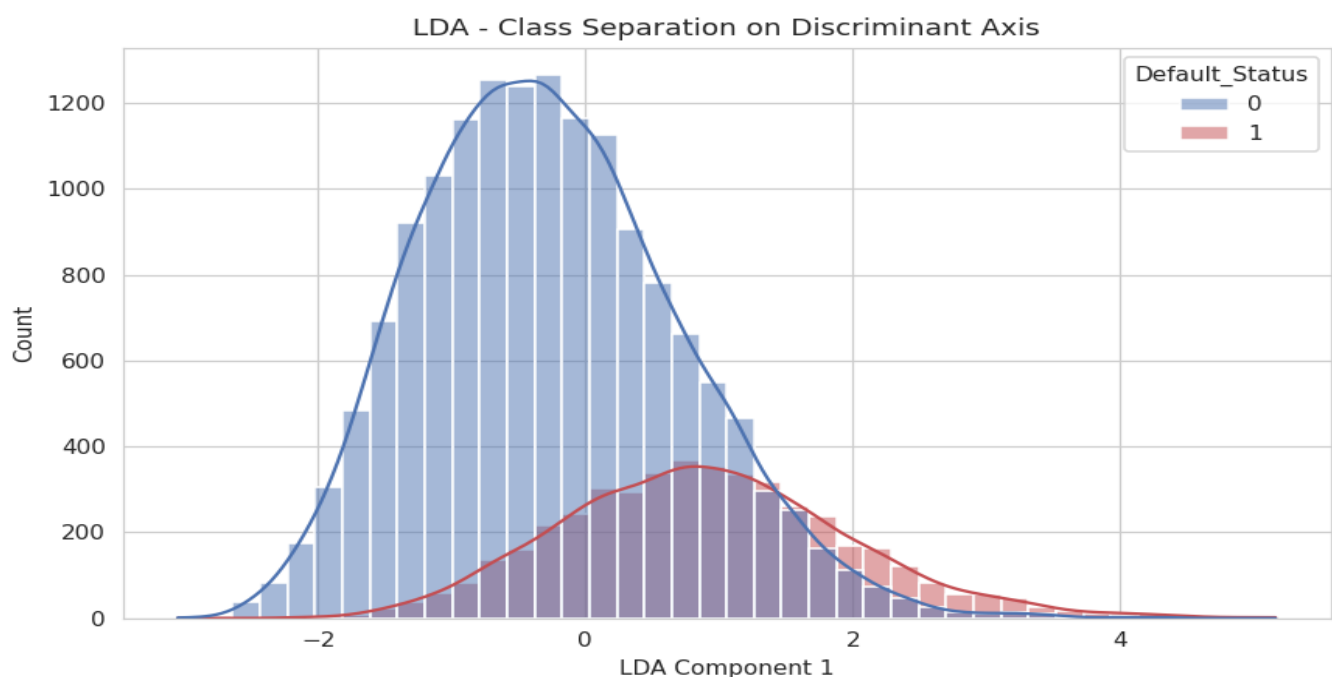


Figure 6: Class separation on the LDA discriminant axis

PCA vs LDA — Comparison

Method	Accuracy	Precision	Recall	F1-Score
All Features (scaled)	0.7995	0.6383	0.2884	0.3973
PCA (95% variance, 9 components)	0.7925	0.6280	0.2314	0.3381
LDA (1 component)	0.7980	0.6280	0.2906	0.3973

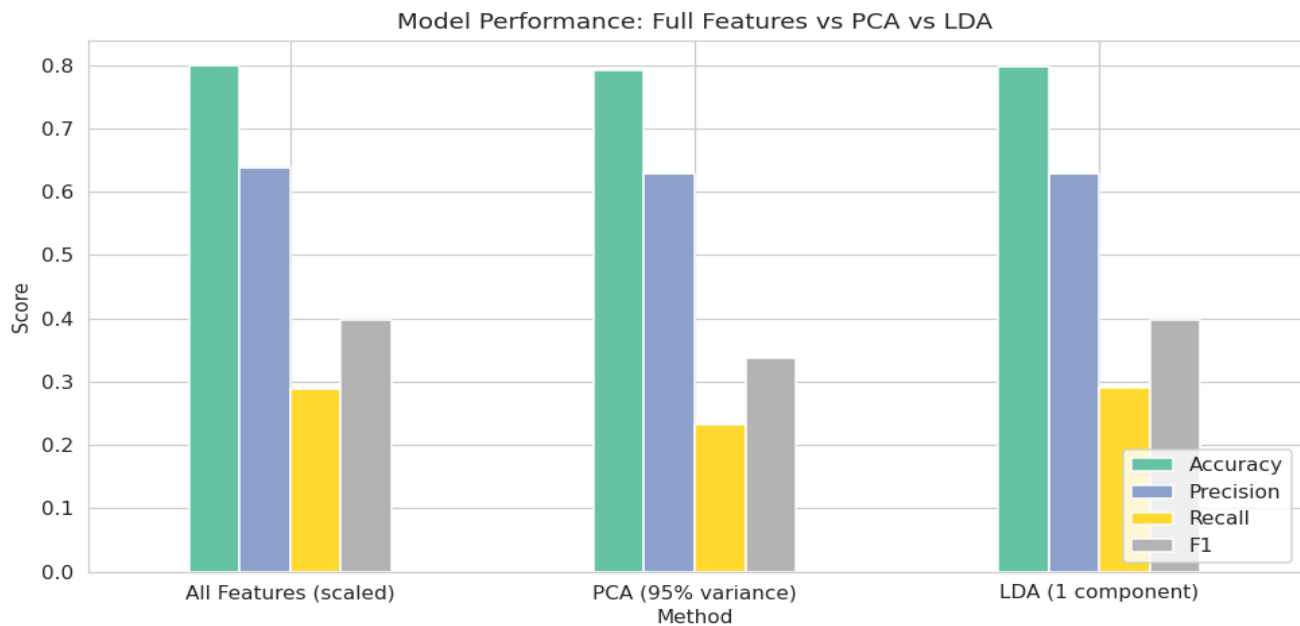


Figure 7: Model performance — full features vs PCA vs LDA

LDA (1 component) matches the full 13-feature model on Accuracy and F1-score while using 92% fewer features than the original set, and outperforms PCA on every metric despite using far fewer dimensions. This is because LDA is supervised — it optimizes directly for class separability — whereas PCA is unsupervised and only preserves variance.

Model Training

- Data Splitting: 80% train / 20% test, stratified on Default_Status, random_state=42
- Model: Logistic Regression trained on the single LDA component
- Hyperparameter Tuning: GridSearchCV over C = [0.01, 0.1, 1, 10], 5-fold CV — best C = 0.1
- Cross-Validation: 5-fold cross-validated accuracy on the full LDA-reduced dataset = 0.7989

Performance Evaluation

Final tuned model (LDA + Logistic Regression, $C=0.1$), evaluated on the held-out 20% test set:

Metric	Value
Accuracy	0.7980
Precision	0.6292
Recall	0.2884
F1-Score	0.3955

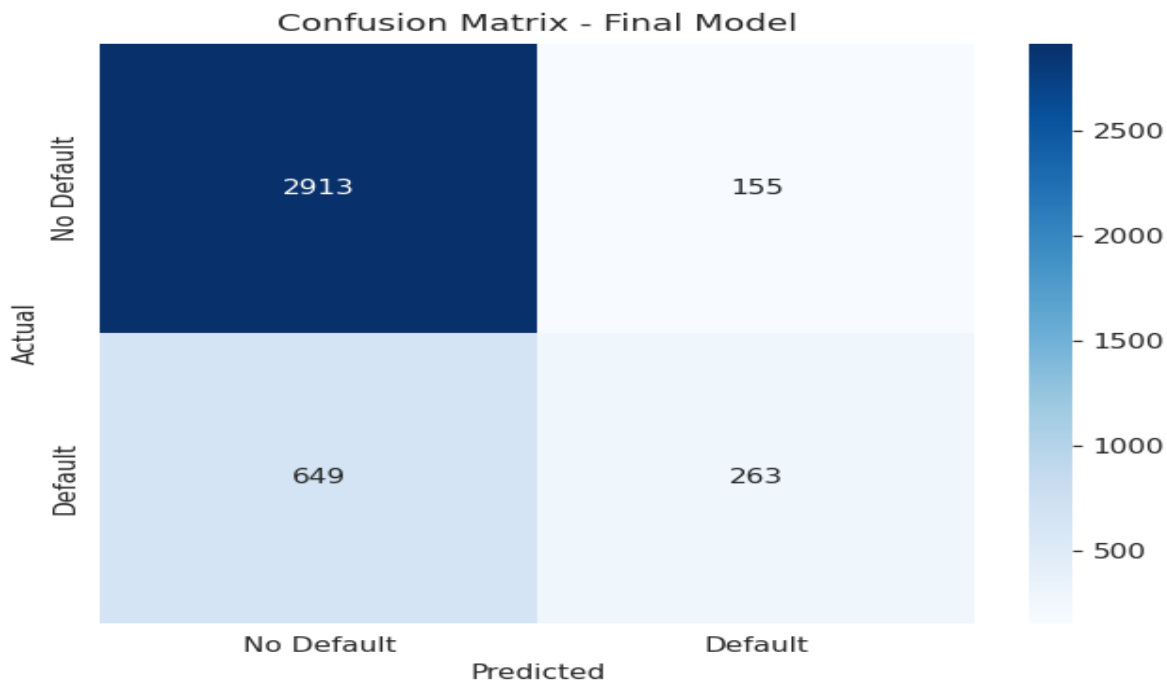


Figure 8: Confusion matrix — final model on the test set

The confusion matrix (2913 true negatives, 155 false positives, 649 false negatives, 263 true positives) shows the model is reliable at identifying non-defaulters but has moderate recall on the minority Default class — expected given the ~23% class imbalance and the use of a single LDA feature.

Results and Discussion

- Credit_Score, Debt_to_Income and Loan_Amount are the strongest, most consistently agreed-upon predictors of loan default across all three feature selection methods.
- LDA reduces 13 engineered features to a single component while matching the full-feature model's accuracy and F1-score, making it the more efficient dimensionality reduction technique for this binary classification task.

- PCA needs 9 components to reach 95% variance and still underperforms LDA, confirming that variance-maximization (unsupervised) is a weaker objective than class-separability-maximization (supervised) when the end goal is classification.
- The tuned model reaches ~80% accuracy but modest recall (0.29) on defaulters — for a real deployment, class-imbalance handling (e.g. class weighting, SMOTE) would likely be a next step to improve default detection.

Model Demonstration — Prediction Results

A new hypothetical customer was passed through the full pipeline (scaling -> LDA transform -> tuned Logistic Regression) to demonstrate real-time prediction:

Input Field	Value
Income	\$45,000
Age	34
Credit_Score	610
Loan_Amount	\$250,000
Existing_Loans	2

Conclusion

This assignment engineered three new risk-related features from raw banking data, compared Filter, Wrapper and Embedded feature selection methods, and evaluated PCA against LDA for dimensionality reduction. LDA proved the more effective reduction technique for this binary classification problem, collapsing 13 features into 1 component without sacrificing predictive performance. The recommended optimal feature set — combining Random Forest importance with cross-method agreement — is: Age, Credit_Score, Debt_to_Income, Existing_Loans, Income, Loan_Amount, Loan_Burden, Loan_to_CreditScore.

Future Improvements

- Address class imbalance (class weights, SMOTE) to improve recall on defaulters.
- Test non-linear models (Random Forest, Gradient Boosting, SVM) on the selected feature set.
- Use SHAP/LIME for explainability alongside the Embedded method's importance scores.

Real-World Applications

- Credit risk scoring at loan origination.
- Early-warning systems for existing loan portfolios.
- Feature-selection pipelines reusable across other banking risk models.